

AEKD – Gestion de Tontine

Rapport d'avancement du MVP — Backend

Ce document résume les travaux réalisés jusqu'à présent sur le backend de l'application de gestion de la tontine AEKD (Amicale des Enfants Kamkop de Douala), conformément au cahier des charges du projet (CLAUDE.md) : Phase 1 (analyse/architecture) et début de la Phase 2 (backend Spring Boot).

Date du rapport	24 septembre 2026
Périmètre	Infrastructure de développement + Backend (User, Role, Member, Auth)
Statut global	En cours — base d'authentification opérationnelle
Dépôt	AEKD-Tontine (branche master)

Sommaire

- 1. Contexte et objectif du MVP
- 2. Mise en place de l'environnement de développement
- 3. Architecture et pile technique du backend
- 4. Modèle de données implémenté
- 5. Migrations de base de données (Flyway)
- 6. Fonctionnalité livrée : authentification (login)
- 7. Anomalie détectée et corrigée
- 8. Tests effectués
- 9. Décisions techniques prises (non métier)
- 10. Points métier encore à valider
- 11. État du code source (git)
- 12. Prochaines étapes

1. Contexte et objectif du MVP

Le projet AEKD-Tontine vise à développer un MVP web permettant de gérer les membres, les séances mensuelles, les cotisations, les paiements, les sanctions, l'épargne individuelle, le fonds commun, les prêts et remboursements de la tontine de l'AEKD. Le développement suit une méthode progressive définie dans le cahier des charges (CLAUDE.md) : analyse et modèle de données, puis backend Spring Boot, puis frontend Angular, puis intégration et tests.

Ce rapport couvre les travaux réalisés à ce jour : la mise en place de l'environnement de développement (base de données) et le début de la Phase 2 (backend) avec les modules d'authentification de base (comptes utilisateurs, rôles, profils membres, connexion).

2. Mise en place de l'environnement de développement

L'objectif initial était de faire tourner PostgreSQL via Docker Compose (fichier `docker-compose.yml` déjà présent dans le dépôt). Plusieurs incidents ont nécessité une bascule vers une installation native.

2.1 Échec de Docker Desktop

Docker Desktop refusait de démarrer (« Docker Desktop is unable to start »). Diagnostic : la mise à jour du sous-système WSL2 échouait avec le code d'erreur `0x80070641` (échec du service Windows Installer), empêchant le backend WSL2 nécessaire à Docker de fonctionner. La réparation nécessitait des droits administrateur et une intervention manuelle sur la machine ; une tentative d'élévation automatique a par ailleurs été bloquée par les garde-fous de sécurité de l'environnement d'exécution (affaiblissement temporaire de l'authentification jugé trop risqué pour être automatisé).

2.2 Bascule vers PostgreSQL natif

Décision (validée avec l'utilisateur) : abandonner Docker pour le développement local et utiliser une installation PostgreSQL native sur Windows (service Windows `postgresql-x64-18`, PostgreSQL 18.6). Une proposition alternative d'utiliser XAMPP a été écartée : XAMPP embarque MySQL/MariaDB et non PostgreSQL, alors que le projet impose explicitement PostgreSQL (CLAUDE.md, section technologies) — changer de moteur aurait constitué une modification d'architecture majeure non demandée.

Un mot de passe superutilisateur oublié a nécessité une réinstallation de PostgreSQL. Une fois le nouveau mot de passe confirmé par l'utilisateur, la base et le rôle applicatifs ont été créés :

```
CREATE ROLE aekd WITH LOGIN PASSWORD '****'; CREATE DATABASE aekd_tontine OWNER aekd;
```

La connexion du rôle applicatif `aekd` à la base `aekd_tontine` a été vérifiée avec succès, et correspond à la configuration attendue par le backend (`application.yml`).

Composant	État
Docker Desktop	Non fonctionnel (bug WSL2 côté OS, hors de portée du projet)
PostgreSQL 18 (service Windows natif)	Opérationnel
Base <code>aekd_tontine</code> / rôle <code>aekd</code>	Créés et vérifiés
<code>docker-compose.yml</code>	Conservé dans le dépôt (réutilisable si Docker est réparé plus tard)

3. Architecture et pile technique du backend

Le backend suit la pile imposée par le cahier des charges : Java 21, Spring Boot 3.5.16, Spring Web, Spring Data JPA, Spring Security, JWT (bibliothèque jjwt), PostgreSQL, Flyway, Bean Validation, Maven. L'architecture en couches Controller → Service → Repository → Database est respectée, avec des modules séparés par domaine métier.

Le squelette de sécurité (JWT stateless) était déjà présent avant cette session : JwtService (génération/validation des tokens), JwtAuthenticationFilter (peuple le contexte de sécurité à partir du token), JwtProperties (configuration externalisée) et SecurityConfig (chaîne de filtres, session stateless, /api/auth/** public). Le rôle applicatif RoleName (ADMIN / TRESORIER / MEMBRE) était également déjà défini, conformément à la section 6 du cahier des charges.

Modules développés au cours de cette session :

Module	Contenu
user	Entités Role et User, repositories associés
member	Entité Member (profil membre, lié 1■1 à User)
auth	Endpoint de connexion (POST /api/auth/login)

4. Modèle de données implémenté

4.1 Table role

Référence fixe des trois rôles applicatifs (ADMIN, TRESORIER, MEMBRE). Ce n'est pas un système de permissions dynamique (explicitement hors périmètre du MVP).

Colonne	Type	Contrainte
id	UUID	Clé primaire
name	VARCHAR(20)	Unique, non nul — ADMIN / TRESORIER / MEMBRE

4.2 Table users

Compte d'authentification, distinct du profil membre.

Colonne	Type	Contrainte
id	UUID	Clé primaire
email	VARCHAR(255)	Unique, non nul — sert d'identifiant de connexion
password_hash	VARCHAR(255)	Non nul — mot de passe chiffré (BCrypt)
enabled	BOOLEAN	Non nul, défaut vrai — permet de désactiver un compte
created_at / updated_at	TIMESTAMPTZ	Non nul — horodatage automatique

4.3 Table user_roles

Table de liaison many-to-many entre users et role (un utilisateur peut porter plusieurs rôles), avec clés étrangères en cascade sur la suppression de l'utilisateur.

4.4 Table member

Profil membre de la tontine, distinct du compte d'authentification. Relation 1■1 avec users (conformément au modèle de données du cahier des charges, section 27), portée côté member via une clé étrangère unique.

Colonne	Type	Contrainte
id	UUID	Clé primaire
user_id	UUID	Unique, non nul — FK vers users, ON DELETE CASCADE
full_name	VARCHAR(255)	Non nul
phone	VARCHAR(30)	Optionnel
join_date	DATE	Non nul — date d'adhésion
active	BOOLEAN	Non nul, défaut vrai
created_at / updated_at	TIMESTAMPTZ	Non nul — horodatage automatique

Note technique : la relation Member → User est unidirectionnelle (Member référence User, pas l'inverse) afin d'éviter un couplage circulaire entre les modules user et member. Le lien inverse (retrouver le Member d'un User) se fait via une requête du repository, pas via une association JPA bidirectionnelle.

5. Migrations de base de données (Flyway)

Version	Nom	Contenu
V1	create_role_and_user_tables	Création des tables role, users, user_roles
V2	seed_roles	Insertion des 3 rôles fixes (ADMIN, TRESORIER, MEMBRE)
V3	create_member_table	Création de la table member

Les trois migrations ont été appliquées avec succès sur la base `aekd_tontine`, et le mapping JPA/Hibernate a été validé contre le schéma réel (`spring.jpa.hibernate.ddl-auto=validate`) : aucune incohérence entre le code et la base.

6. Fonctionnalité livrée : authentification (login)

Endpoint `POST /api/auth/login` implémenté : vérifie l'email et le mot de passe (comparaison BCrypt) contre la table `users`, puis génère un JWT signé contenant l'identité et les rôles de l'utilisateur, réutilisant le service `JwtService` déjà présent dans le projet.

Comportement vérifié par test :

Scénario	Résultat HTTP	Comportement
Email + mot de passe corrects	200 OK	Retourne un JWT, l'email et la liste des rôles
Mot de passe incorrect	401 Unauthorized	Message générique (aucune fuite d'information)
Email inconnu	401 Unauthorized	Même message générique que ci-dessus
Champs vides / invalides	400 Bad Request	Erreur de validation (Bean Validation)

Le compte utilisé pour ce test de bout en bout était temporaire (inséré directement en base, hors du code applicatif) et a été supprimé immédiatement après la vérification : aucune donnée de démonstration n'a été laissée en base à ce stade — la création des comptes de démonstration (section 32 du cahier des charges) reste une tâche à part entière, non encore réalisée.

7. Anomalie détectée et corrigée

En testant les cas d'échec du login, toutes les erreurs (mot de passe incorrect, validation invalide, etc.) renvoyaient un code 403 avec une réponse vide au lieu du code attendu (401 ou 400). Cause : la route interne `/error`, utilisée par Spring pour construire la réponse d'erreur, n'était pas autorisée dans la configuration de sécurité ; Spring Security interceptait ce transfert interne et le remplaçait par un 403 générique, masquant la vraie erreur.

Correction : ajout de `/error` à la liste des routes publiques dans `SecurityConfig`. Après correction, les codes HTTP et les messages d'erreur retournés sont corrects (voir tableau section 6).

8. Tests effectués

Conformément à la méthode définie dans le cahier des charges (compiler, tester, corriger, vérifier les migrations et les endpoints), les vérifications suivantes ont été effectuées après chaque étape :

- Compilation Maven (`mvn compile`) après chaque ajout de code — sans erreur.

- Démarrage complet du backend après chaque migration — sans erreur, connexion base et mapping JPA validés.
- Tests automatisés (JUnit, mvn test) sur le service d'authentification : 3 scénarios (connexion réussie, mot de passe erroné, email inconnu), exécutés dans une transaction annulée automatiquement après chaque test (aucune donnée résiduelle en base).
- Tests HTTP réels (curl) sur l'endpoint `/api/auth/login` pour les 4 scénarios listés en section 6, avant et après correction de l'anomalie `/error`.
- Vérification manuelle du contenu de la base (tables `role`, `users`, `user_roles`, `member`) via `psql` après chaque migration.

Résultat de la dernière exécution des tests automatisés : **Tests run: 3, Failures: 0, Errors: 0**
— **BUILD SUCCESS.**

9. Décisions techniques prises (non métier)

Conformément au principe de non-invention des règles métier (CLAUDE.md, section 2), les choix ci-dessous sont des décisions purement techniques d'implémentation, sans impact sur les montants, seuils ou règles de fonctionnement de la tontine :

- Identifiants primaires en UUID pour toutes les entités (plutôt que des entiers auto-incrémentés), pour éviter des identifiants devinables côté API.
- Relation User ↔ Role en many-to-many (cohérent avec le JwtService existant, qui gère déjà une liste de rôles par utilisateur).
- Relation Member → User unidirectionnelle, portée par Member, pour préserver l'indépendance des modules.
- User ne référence pas encore Member (la contrainte sera ajoutée quand ce sera nécessaire) : un compte peut exister sans profil membre à ce stade du développement.
- Erreurs d'authentification volontairement génériques (« Email ou mot de passe incorrect ») pour ne pas révéler si un email existe en base.
- Aucun contrôleur/endpoint créé au-delà du strict nécessaire pour valider chaque étape (pas de gestion des utilisateurs, pas de CRUD Member) — ces fonctionnalités restent à développer.

10. Points métier encore à valider

Le fichier `docs/decisions.md` du dépôt recense l'ensemble des décisions métier en attente de validation (montants de « Voir bébé » et « Banque / Épargne », délais de retard, montants de sanctions, règles précises des prêts, etc.). Aucune de ces décisions n'a été impactée par les travaux décrits dans ce rapport, qui se limitent à l'infrastructure technique (comptes, rôles, profils, connexion).

11. État du code source (git)

Le dépôt contient un seul commit à ce jour (« feat: initialize git repo and make backend startable »). Tous les travaux décrits dans ce rapport sont présents dans l'arborescence de travail mais **n'ont pas encore été commités** :

```
Modifié : backend/.../security/SecurityConfig.java Nouveau : backend/.../auth/
(LoginRequest, LoginResponse, AuthService, AuthController) Nouveau : backend/.../user/
(Role, User, RoleRepository, UserRepository) Nouveau : backend/.../member/ (Member,
MemberRepository) Nouveau : backend/src/main/resources/db/migration/ (V1, V2, V3) Nouveau
: backend/src/test/ (AuthServiceTest)
```

Un commit n'a pas été créé automatiquement, conformément à la consigne de ne committer que sur demande explicite.

12. Prochaines étapes

Dans l'ordre recommandé par le cahier des charges (développement progressif, sans tout générer en une seule fois) :

- Session (concept central, section 7) : entité de séance mensuelle, socle des cotisations.
- ContributionDefinition / ContributionPeriod : configuration des cotisations, distincte de leur application par séance.
- Participants et bénéficiaires des cotisations.
- Déclaration et validation des paiements, justificatifs.
- Sanctions (règles puis application).
- Fonds commun et épargne individuelle.
- Prêts, politiques de prêt et remboursements.
- Tableaux de bord et journal d'audit.
- Frontend Angular (une fois une base d'API suffisante disponible).